

Six-Day Runway

What to build with your own hands before 21 September, if today you know basic LWC and basic Apex. Six days, one drill each, every drill ending in something that runs.

github.com/esron-compliance-command/cc-react-lwc-starter

PREPARED

15 September 2026

FOR

Developers and QA joining the project

PROJECT START

21 September 2026, Day 1

DELIVERY WINDOW

Approximately 2.5 months

READ THIS FIRST

Eleven mandatory topics do not fit into six days at equal depth, and pretending otherwise is how people arrive having skimmed everything and practised nothing. So this plan triages. The ordering below is not by importance to the product – it is by *how early a gap in it stops you from committing code*.

BLOCKS D1

React, TypeScript, the React-in-LWC bridge, Git/PR flow, VS Code + sf CLI

You cannot pick up a ticket without these. The bridge in particular is the one thing no tutorial will teach you, because it is specific to how this platform hosts React.

BLOCKS W2

Selector / Service separation, sharing & FLS, Experience Cloud LWR

You will write your first Apex against an existing pattern and your first component onto an existing site. Know the shape; the specifics are learnable in context.

IN REVIEW

SOLID, DRY, KISS, YAGNI, cyclomatic complexity

Real, and non-negotiable in review – but they are judgement applied to code you are writing, not a syllabus to finish beforehand. One careful read, then apply them.

BEFORE DAY 1 – ABOUT AN HOUR

- **Node 20 LTS** and npm. Verify with `node -v`. Everything React in this project is built locally with esbuild – there is no CDN at runtime.
- **Salesforce CLI** (`sf`, not the retired `sfdx`), plus VS Code with the Salesforce Extension Pack, ESLint and Prettier.
- **Your own free Developer Edition org** – developer.salesforce.com/signup. Do not wait for project org access; every drill here runs in your own org.
- **A GitHub account with SSH keys** and one throwaway practice repo you are willing to break.

THE CODE TO REFER TO

github.com/esron-compliance-command/cc-react-lwc-starter is a runnable miniature of the architecture you are joining: a React + TypeScript grid, bundled with esbuild, hosted inside an LWC, backed by one Apex controller that is generic over the object. Clone it, deploy it to your own Developer org, sort a column, then read the four files that made that happen. About twenty minutes.

It is verified, not aspirational: all 20 components deploy from a single command, 12 Apex, 9 React and 4 LWC tests pass, and CI is green from a clean clone. Its README is Day 4 and Day 5 of this plan in executable form. Start there on Friday rather than from a blank folder.

PUBLIC REFERENCE, FOR THE PARTS A TUTORIAL CAN TEACH

- **trailheadapps/lwc-recipes** – the `libsD3`, `libsChartjs` and `libsFullCalendar` recipes are the `loadScript` + static resource + `lwc:dom="manual"` pattern, the nearest public thing to our bridge. Eight `apex*` recipes cover wire vs imperative. There is no React example in it – that half is ours.
- **trailheadapps/apex-recipes** – Apex patterns, including security and dynamic SOQL.
- **trailheadapps/ebikes-lwc** – a complete sample app with an Experience Cloud site.

LOCAL GREEN PROVES LESS THAN YOU THINK

Building the starter turned up three failures that passed every local check and only appeared on a real deploy: `sort` is a reserved identifier in Apex; a custom-metadata record with its XML namespaces declared in the wrong place fails as `UNKNOWN_EXCEPTION` with *no* component detail; and a missing `.forceignore` line makes the deploy try to compile your Jest test. All three are written up in the README. Deploy early and often.

THE SIX DAYS

01

TUE 15 SEP

Git until conflicts stop being scary

GIT & GITHUB · VS CODE

The listed items – branches, commits, PRs, merge conflicts, reviews – are not five topics. They are one workflow, and the only part that reliably goes wrong under deadline is the conflict. Manufacture some today, deliberately, while nothing is at stake.

DRILL

1. In your practice repo, create `feature/a` and `feature/b` from `main`. Edit *the same lines of the same file* on both.
2. Merge A. Then merge B and resolve the conflict by hand in VS Code – read both sides, keep the right one. Do this three times until the markers stop looking alarming.
3. Open a PR from a third branch. Review your own diff line by line and leave real comments on it. Then `git rebase main` onto a moved main and resolve again.
4. Learn the two recoveries you will actually need: `git stash` and `git reflog`. Knowing reflog exists is the difference between a lost afternoon and thirty seconds.

LOCAL REFS ARE A CACHE

Run `git fetch` before you conclude a commit is or is not on a branch. A stale local ref has cost this team a wrong answer in a status call more than once.

DONE WHEN

You can resolve a three-file conflict without asking anyone, and explain what rebase did to your commit hashes.

02

WED 16 SEP

TypeScript, and the JavaScript underneath it

TYPESCRIPT · MODERN JS

Most LWC work can be done in fairly plain JavaScript. React code in this project cannot: it is `.tsx` throughout, with typed props, typed API envelopes and discriminated unions for state. The gap for an LWC developer is usually not React – it is immutability and types.

PRACTISE

- ES modules, destructuring, rest/spread, optional chaining, `??`, `async/await`, `Promise.all`.
- **Immutable updates.** `[...rows, row]`, `rows.map(...)`, `{...obj, key: value}` – never `push` or in-place mutation. React re-renders on new references, so a mutated array is an invisible bug, not an error.
- `interface` vs `type`, unions, generics (`Array<T>`, `Promise<T>`, your own `<T>`), `import type`, `strict` null handling.

DRILL

Write a plain-Node TypeScript module – no React yet – that models a paged list response and transforms it:

```
export interface ListEnvelope<T> {
  rows: T[];
  totalCount: number;
  pageNumber: number;
}

export function mapRows<T, R>(
  envelope: ListEnvelope<T>,
  fn: (row: T) => R
): ListEnvelope<R> {
  return { ...envelope, rows: envelope.rows.map(fn) };
}
```

Then make `npm run tsc --noEmit` pass with `"strict": true`. Fight the errors; do not reach for `any`. That fight is the learning.

DONE WHEN

You can read a generic signature out loud and say what it guarantees, and `tsc --noEmit` is clean without a single `any`.

03

THU 17 SEP

React, built rather than watched

REACT · COMPONENTS, PROPS, STATE, HOOKS

Scaffold with `npm create vite@latest -- --template react-ts` and build one non-trivial thing. Not a to-do list – build the shape of what we actually ship: a data table.

DRILL — A TABLE AGAINST A FAKE ASYNC API

- A `fakeApi.ts` that returns a `Promise` after 400ms and does the sorting, filtering and paging *itself*. This matters: in our grid the server owns all three, and the client only asks. Practising client-side sorting teaches the wrong instinct.
- Server-driven sort, search (debounced 300ms), filters, pagination. Loading, empty and error states as first-class renders, not afterthoughts.
- A custom hook, `useListData(config)`, owning every piece of that state – components render, the hook decides.
- Controlled inputs, correct `key` props, `useEffect` with a cleanup function, `useCallback` / `useMemo` only where you can say why.

STRICTMODE DOUBLE-INVOKES

Our bundle mounts inside `<StrictMode>`. In development every effect runs twice on purpose. If your fetch fires twice, that is not a bug to suppress – it is telling you the effect is not idempotent or is missing its cleanup. Track a request sequence and ignore stale responses.

DONE WHEN

Typing in search does not fire a request per keystroke, changing a filter resets to page one, and a slow response that lands after a newer one does not overwrite the screen.

04

FRI 18 SEP

The bridge — React running inside an LWC

THE SINGLE HIGHEST-VALUE DAY ON THIS LIST

This is the piece nobody arrives already knowing, and it is the whole architecture. React cannot `import` from `@salesforce/apex` – it is not inside an LWC module graph. So a thin LWC host loads the compiled bundle from a static resource and *injects* a set of promise-returning functions. React calls those. That injected object is the entire contract between the two worlds.

DRILL – BUILD THE WHOLE LOOP YOURSELF

1. Bundle yesterday's React app with esbuild into one self-contained IIFE that attaches a single global, then deploy the output as a static resource.
2. Write an LWC host that loads it and mounts it into a manual DOM node:

```

<!-- host.html -->
<div class="mount" lwc:dom="manual"></div>

// host.js
import { loadScript, loadStyle } from
'lightning/platformResourceLoader';
import myBundle from '@salesforce/resourceUrl/myBundle';
import fetchList from '@salesforce/apex/MyController.fetchList';

async renderedCallback() {
  if (this.loaded) return;
  this.loaded = true;
  await loadScript(this, myBundle + '/app.js');
  window.MyApp.configure({
    fetchList: (req) => fetchList({ requestJson:
JSON.stringify(req) })
                                .then(JSON.parse)
  });
  window.MyApp.mount(this.template.querySelector('.mount'), {
objectApiName: 'Account' });
}

disconnectedCallback() {
  window.MyApp?.unmount(this.template.querySelector('.mount')); }

```

On the React side, `createRoot(el).render(...)`, keep the roots in a `WeakMap` keyed by element so unmount can find them, and read the bridge from exactly one module so a test can swap in a fake.

ALSO UNDERSTAND TODAY

- **Why `lwc:dom="manual"`**: synthetic shadow DOM will not let React own a node otherwise.
- **Why no CDN**: Experience Cloud pages can be public or on locked-down networks. React and ReactDOM are bundled *in*.
- **Lightning Web Security** sanitizes the DOM. Inline `<svg>` in a community flashes and then vanishes – icons ship as `<img>` from a static resource. Expect a class of "works locally, gone in the site" bugs and know where they come from.
- Payloads cross the boundary as JSON strings in both directions. Practise `JSON.stringify` in, `JSON.parse` out.

DONE WHEN

A React table is rendering real rows from your own Apex, inside an LWC, in your own Developer org – and you can explain, without notes, why React never imports Apex directly.

WORTH KNOWING BEFORE YOU STUDY

This repository does **not** use the `fflib` library. There are no `fflib_*` classes in it. The same separation is implemented in-house: an `Application` singleton handing out services and selectors, an abstract `ASelector` base, ~64 `*Selector` classes, `*Service` classes, and `*TriggerHandlers` behind 144 triggers. Learn the *patterns* – selector, service, domain, unit of work – because they are exactly what review will hold you to. Just do not spend the week memorising an API this codebase does not call.

PRACTISE

- **Separation of concerns.** Query in a selector. Business rules in a service. Never SOQL inside a controller method, never business logic in a trigger body.
- **Sharing keywords** and what each really does. Know the trap: `USER_MODE / SYSTEM_MODE` control FLS and CRUD, *not* record sharing. A `with sharing` class on a `Private` object returns zero rows to a community user no matter what access level you pass.
- **Dynamic SOQL, safely.** Field names drawn only from a configuration allowlist, never from raw user input, then:

```
Database.queryWithBinds(soql, binds, AccessLevel.USER_MODE);
```

That throws on a field the running user cannot read, rather than silently leaking it. Note that the platform refuses `WITH SECURITY_ENFORCED` alongside an explicit access level, and refuses it on `COUNT()` entirely – `USER_MODE` covers both.

- **Describe and metadata-driven design.** `Schema.describeSObjects`, `fields.getMap()`, resolving an object or field by API name at runtime. Our grid is generic over any object because its columns come from custom metadata, not from code.
- **AuraHandledException.** Always `e.setMessage('...')` as well as constructing it – the constructor alone leaves the message empty and the user sees only "An error occurred."

DRILL

Write one `@AuraEnabled` controller that lists records for *any* object passed as a string, with columns driven by a custom metadata type you define, searching/sorting/paging in SOQL, running in `USER_MODE`, returning a serialized envelope. Then point yesterday's React table at it. That is, in miniature, the thing you will be extending in October.

DONE WHEN

Your controller works against two different objects without a code change, and removing a field from your permission set makes it throw rather than quietly return blanks.

06

SUN 20 SEP

Put it on a site, then prove it works

EXPERIENCE CLOUD LWR · TESTS · PRINCIPLES

EXPERIENCE CLOUD / LWR

- Create an LWR site in your Developer org. Learn the loop: build → deploy static resource and LWC → `sf community publish`. Skip the publish and the site keeps serving the old bundle while the deploy still reports success. This catches everyone once.
- Understand that deployed does not mean visible. A component must be placed on the page in Experience Builder, and a new `@AuraEnabled` class needs to be in the community permission set, or guests and customers see nothing.
- Guest vs authenticated context, sharing sets, and why a community user's record access is the first thing to check when a list is mysteriously empty.

TESTS — NOT OPTIONAL HERE

- **Vitest + React Testing Library** for React: query by role and label, drive with `userEvent`, inject a fake bridge instead of mocking network.
- **Jest (sfdx-lwc-jest)** for the LWC host.
- **Playwright** for end-to-end. On this team every fix ships with an e2e test in the same PR. Install it and write one real test against your own site so the API is familiar before it is urgent.

PRINCIPLES — ONE HONEST READ

DRY, KISS, YAGNI, SOLID, and cognitive complexity are review vocabulary. Spend an hour reading, then go back to yesterday's controller and ask: what does this method do that its name does not say? How many branches deep is it? Which part would I have to copy to support a second object? Refactor once. That hour teaches more than a day of articles.

DONE WHEN

Your React grid is live on your own LWR site, a Playwright test drives it and passes, and your unit tests run green from the command line.

THINGS THIS PLATFORM DOES THAT WILL SURPRISE YOU

Each of these has already cost this team real time. They are cheap to read now and expensive to rediscover in October.

Never hardcode the namespace The package namespace is <code>cocmd</code> ; the QA org aliases it to <code>coqa</code> . A serialized SObject carries whichever prefix the <i>running</i> org uses, so <code>result.cocmd__Status__c</code> is blank in QA. Resolve fields namespace-agnostically.	A new field is invisible until FLS says otherwise Deploy a field without adding it to a permission set and nobody sees it – including admins. A field-level-security gap in <code>USER_MODE</code> can empty an entire grid, not just one column.
Deployed ≠ visible Community pages are hand-built. A component that deploys perfectly still shows nothing until it is placed on the page and its Apex is in the community permission set. Verify render <i>and</i> data as a real community user.	Lightning Web Security rewrites your DOM Inline SVG is sanitized away. Some globals are proxied. Anything that reaches around the framework to touch <code>document</code> should be assumed broken inside a community until proven otherwise.
Read the error, not the code Diagnose from the exception text first. A single-row SOQL assignment that returns zero rows is a live, recurring bug class here, and the message names it immediately.	Test data is production data Never create records named "test" or "e2e" in a shared org. Name things the way a real customer would – demos happen without warning.

DO NOT SPEND THIS WEEK ON

~~Next.js~~

~~Redux~~

~~React Router~~

~~Tailwind~~

~~class components~~

~~Aura~~

~~fflib library API~~

~~webpack config~~

~~GraphQL~~

~~React Native~~

Our bundle is plain React 18 with hooks, esbuild, and hand-written CSS. There is no router, no global state library and no CSS framework in it. A week spent on Next.js is a week not spent on the bridge.

You are not becoming developers — you are becoming able to file a useful bug

- **Git/GitHub:** read a PR, find which branch a fix is on, and check whether it was merged before the build you are testing was cut. Half of "it regressed" turns out to be "the fix was not in that build".
- **Where the code runs:** a React grid inside an LWC on an LWR site. So "the table is wrong" splits three ways — the React rendering, the Apex data, or the site configuration. Saying which one you saw saves a day.
- **Browser devtools:** the console error and the failing network call, captured in the ticket. The verbatim error text is usually the whole diagnosis.
- **Environments:** know which org you are in and which package version it runs. An installed org can be weeks behind the branch.

READY FOR 21 SEPTEMBER WHEN YOU CAN SAY YES TO ALL OF THESE

- 01 I resolved a merge conflict today without help, and I know what `git reflog` is for.
- 02 I wrote TypeScript with a generic in it and got `tsc --noEmit` clean under `strict`, with no `any`.
- 03 I built a React table with server-driven sort, search, filter and paging, and its state lives in a custom hook.
- 04 I have React rendering inside an LWC in my own org, loaded from a static resource, calling my own Apex through an injected bridge.
- 05 I can explain why React cannot import `@salesforce/apex` directly.
- 06 I wrote an Apex controller that is generic over the object, driven by custom metadata, querying in `USER_MODE`.
- 07 I know the difference between what `USER_MODE` enforces and what record sharing enforces.
- 08 My component is live on an LWR site I published, and I know what happens if I forget to publish.

09 I have a Playwright test that drives my own UI and passes.

10 My questions on Day 1 are about *our* architecture, not about what React is.